Đồ án 2

ĐỀ CƯƠNG NGHIÊN CỨU

TÊN ĐỀ TÀI
Mini-NPU: NPU RISC-V lấy cảm hứng từ kiến trúc CoralNPU với MAC engine int8 tách rời, được kiểm chứng bằng môi trường UVM tự xây dựng và chứng minh khả năng chạy mô hình AI int8 (MNIST)

Mục	Thông tin
Sinh viên / Nhóm	Nguyễn Hoàng Công Tấn / Nguyễn Thế Bảo
MSSV	23161064 / 23144002
Chương trình / Khóa	Công nghệ kỹ thuật Điện tử - Viễn thông, lớp 23161FIE
Giảng viên hướng dẫn	TS. Phạm Văn Khoa
Lĩnh vực nghiên cứu	Bộ tăng tốc AI cho thiết bị biên (NPU RISC-V), thiết kế RTL và kiểm chứng UVM
Thời gian thực hiện	16 tuần (12/10/2026 – 31/01/2027)
Ngày nộp	11/11/2026

Logic nghiên cứu bắt buộc

Giai đoạn	Câu hỏi cốt lõi	Bằng chứng yêu cầu
1. Vấn đề kỹ thuật	Chính xác thì điều gì sai, bị hạn chế, tốn kém, chậm, thiếu chính xác, không an toàn hoặc chưa được giải quyết về mặt kỹ thuật?	Vấn đề được định lượng, ràng buộc, thước đo, câu hỏi nghiên cứu
2. Công trình liên quan	Các nghiên cứu hoặc hệ thống hiện có đã giải quyết vấn đề này như thế nào?	So sánh có phản biện, kết quả, hạn chế, khoảng trống nghiên cứu
3. Ý tưởng đề xuất	Ý tưởng mới hoặc được điều chỉnh nào được đề xuất để lấp khoảng trống?	Khái niệm hệ thống, kiến trúc, tính mới, giả thuyết
4. Phương pháp luận	Ý tưởng sẽ được hiện thực và đánh giá khách quan như thế nào?	Kế hoạch hiện thực, baseline, thí nghiệm, thước đo, phân tích
5. Kết quả kỳ vọng	Những kết quả đo được và sản phẩm bàn giao nào được kỳ vọng?	Hiệu năng mục tiêu, đóng góp, prototype/báo cáo/demo

Chuẩn viết khuyến nghị

- Dùng thuật ngữ kỹ thuật chính xác và các chỉ số hiệu năng đo được thay cho những nhận định chung chung như “tốt hơn”, “nhanh hơn” hay “hiệu quả hơn”.
- Dùng các nguồn được bình duyệt và đáng tin cậy về mặt kỹ thuật. Ưu tiên công trình gần đây cho hiện trạng kỹ thuật, đồng thời giữ lại các tài liệu nền tảng khi cần.
- Với mỗi nhận định quan trọng, đưa ra bằng chứng qua trích dẫn, phép đo, dữ liệu benchmark hoặc một giả định được nêu rõ.
- Định nghĩa mọi từ viết tắt ở lần dùng đầu tiên và dùng nhất quán ký hiệu, đơn vị, tên dataset, đặc tả phần cứng và điều kiện đánh giá.
- Giải pháp đề xuất phải trực tiếp giải quyết khoảng trống nghiên cứu rút ra từ các công trình liên quan; các thước đo đánh giá phải phản ánh trực tiếp các vấn đề kỹ thuật nêu ở Phần 1.

Độ dài đề xuất

Thành phần	Độ dài đề xuất
Các phần chính 1-5	6-10 trang
Hình / Bảng	Tùy theo nhu cầu làm rõ kỹ thuật
Tài liệu tham khảo	Thông thường từ 10 nguồn đáng tin cậy trở lên cho một đề tài hướng nghiên cứu
Phụ lục	Tùy chọn; chi tiết hiện thực, bảng mở rộng, bảng rủi ro, v.v.

1 VẤN ĐỀ KỸ THUẬT / MÔ TẢ BÀI TOÁN

Xác định bài toán kỹ thuật bằng các thuật ngữ đo được và có cơ sở kỹ thuật.

1.1 Bối cảnh và ngữ cảnh kỹ thuật

Suy luận machine learning đang chuyển từ trung tâm dữ liệu sang các thiết bị chạy pin và bị giới hạn chi phí như thiết bị đeo, cảm biến và thiết bị nhà thông minh. Trên các thiết bị này, khối lượng tính toán bị chi phối bởi phép nhân-tích lũy (multiply-accumulate, MAC) trên số nguyên nhỏ, và lượng tử hóa sau huấn luyện xuống số nguyên 8 bit (int8) giữ được độ chính xác trong khi cho phép suy luận chỉ dùng số học số nguyên [1]. Các bộ tăng tốc chuyên dụng khai thác tính đều đặn này: Tensor Processing Unit (TPU) dùng một matrix unit 8 bit lớn [2],

Eyeriss giảm năng lượng di chuyển dữ liệu bằng một dataflow riêng [3], và khảo sát trong [4] cho thấy hiệu năng và năng lượng bị chi phối bởi mức tái sử dụng dữ liệu và mức độ các đơn vị tính toán được giữ bận. Nhận định rằng phép tính tốn năng lượng ít hơn nhiều so với việc di chuyển dữ liệu [5] khiến các bộ nhớ gắn chặt (tightly-coupled memory) nhỏ và cổng toán hạng rộng trở nên hấp dẫn cho một neural processing unit (NPU) ở thiết bị biên.

Các dự án mã nguồn mở giúp việc nghiên cứu những kiến trúc này trở nên khả thi. Coral NPU của Google kết hợp một scalar core RISC-V nhỏ với các vector unit, matrix unit và bộ nhớ gắn chặt [6]; Gemmini sinh ra các bộ tăng tốc systolic-array gắn vào các core RISC-V [7]. Dự án này đi theo kiến trúc của Coral NPU ở quy mô mà một nhóm hai người có thể thiết kế, kiểm chứng và giải thích trọn vẹn. Một core RV32IM 5 tầng (với các extension Zicsr, Zifencei và Zbb [8]) được mở rộng thêm bốn custom instruction là LOAD_WEIGHT, LOAD_INPUT, COMPUTE và READ_RESULT, đẩy các command qua một FIFO 8 phần tử tới một MAC engine outer-product int8 chạy tách rời với N×M×K = 4×4×4 (16 processing element, mỗi PE là một dot product 4 phần tử, 64 MAC mỗi cycle). Core và engine dùng chung một bộ nhớ dữ liệu hai cổng 64 KB (DTCM). Đề tài thuộc Computer Engineering vì nó gộp mở rộng tập lệnh, vi kiến trúc, thiết kế hệ thống bộ nhớ, firmware bare-metal và kiểm chứng phần cứng trong một hệ thống, đúng chủ đề đồng thiết kế phần cứng/phần mềm của tính toán chuyên dụng [9].

Một NPU chỉ hữu ích khi toàn bộ chuỗi hoạt động cùng nhau: giải mã lệnh và đẩy command, hàng đợi command, máy trạng thái và cơ chế xử lý lỗi của engine, các lần đọc bộ nhớ cấp dữ liệu cho array, các accumulator, việc đọc kết quả về, và firmware điều phối tất cả. Suy luận lượng tử hóa cũng phải chính xác ở mức số nguyên thì mới đáng tin, vì sai thứ tự command, sai kiểm tra dải địa chỉ, sai wrap của accumulator hoặc sai bước requantization đều cho ra dự đoán sai mà không báo lỗi. Các thiết kế mở và thiết kế của sinh viên thường được trình bày bằng vài directed test hoặc waveform; ít thiết kế đi kèm bằng chứng kiểm chứng có hệ thống và một lần chạy mô hình đầu-cuối khớp từng bit với một golden model độc lập. Thực hành công nghiệp kiểm chứng các khối như vậy bằng Universal Verification Methodology (UVM), golden reference model, assertion và functional coverage [10][11], và dự án tự xây môi trường đó.

Workload minh họa là một multilayer perceptron (MLP) kích thước 784-64-32-10 trên tập chữ số viết tay MNIST [12], được lượng tử hóa int8 với requantization số nguyên giữa các lớp [1]. Nó đủ nhỏ để nằm gọn trong DTCM 64 KB, nhưng vẫn dùng đến mọi phần của NPU: nạp weight và input, nhân ma trận theo tile, tích lũy, đọc kết quả về và requantization trong firmware.

1.2 Phát biểu bài toán kỹ thuật

Bài toán kỹ thuật mà dự án giải quyết là: chưa có một NPU RISC-V nhỏ, được tài liệu hóa đầy đủ, xây dựng trên kiến trúc kiểu CoralNPU, mà tính đúng đắn của nó được thiết lập bằng kiểm chứng có hệ thống và khả năng chạy mô hình AI của nó được chứng minh đầu-cuối so với một golden model độc lập. Thiếu bằng chứng như vậy, một thiết kế NPU không thể được tin cậy với workload thật. Vì vậy dự án phải (i) thiết kế NPU (mở rộng core, custom instruction, command FIFO, MAC engine, bộ nhớ hai cổng), (ii) kiểm chứng engine, core và toàn SoC bằng một môi trường UVM tự xây dựng theo các tiêu chí đóng đo được, và (iii) chạy một MLP đã lượng tử hóa trên nó với logit khớp bit-exact với golden model.

1.3 Ràng buộc kỹ thuật và điều kiện vận hành

Ràng buộc / Điều kiện	Giá trị hiện tại hoặc yêu cầu	Ý nghĩa kỹ thuật
Tài nguyên tính toán	QuestaSim 10.7c (UVM 1.2, DPI-C, coverage); Verilator 5.020 và RISC-V GNU toolchain trong WSL2 (Ubuntu 24.04); Icarus Verilog 12 và GitHub Actions CI; Python 3 với numpy và PyTorch	QuestaSim chạy flow UVM. Bộ tính năng của nó (randomize, covergroup, DPI-C) đã được kiểm ở tuần 1. Verilator chỉ dùng để lint.

Ràng buộc / Điều kiện	Giá trị hiện tại hoặc yêu cầu	Ý nghĩa kỹ thuật
Bộ nhớ / Lưu trữ	ITCM 16 KB; DTCM 64 KB (4 bank × 32 bit); WBUF và IBUF mỗi cái 256 × 128 bit; command FIFO 8 phần tử; KMAX = 1024 phần tử; vùng dữ liệu của model 56,816 B	Toàn bộ MLP nằm vừa trong DTCM, nên không cần DMA hay bộ nhớ ngoài.
Độ trễ / Yêu cầu thời gian thực	Không có deadline thời gian thực. Mỗi command của engine tốn K/4 + 2 cycle bận (chặn trên K/4 + 3). Số cycle trên mỗi lần suy luận được báo cáo như một phép đo, không đặt trước mục tiêu.	Cho thấy bao nhiêu thời gian chạy nằm trong engine và bao nhiêu trong firmware scalar.
Năng lượng / Công suất	Không phải mục tiêu của dự án. Không dùng synthesis, thư viện công nghệ hay ước lượng công suất.	Phần công suất và năng lượng để dành cho luận văn tốt nghiệp.
Độ chính xác / Chất lượng	MLP int8: 97.43 % trên 10,000 ảnh test MNIST (model float 97.48 %); logit của RTL khớp bit-exact với golden model trên 1,000 ảnh đầu	Tiêu chí thành công là bit-exact chứ không phải accuracy; accuracy cho thấy lượng tử hóa không phải yếu tố giới hạn.
Truyền thông / Băng thông	DTCM port A 32 bit đọc/ghi (core); port B 128 bit chỉ đọc (engine), mỗi cycle một quad gồm bốn word	Port B cấp một entry WBUF/IBUF mỗi cycle, giữ cho PE array 64 MAC luôn có dữ liệu.
Chi phí / Diện tích / Mức dùng tài nguyên	N×M×K = 4×4×4: 16 PE dot4 (64 bộ nhân int8); 8×8 chỉ làm nếu còn thời gian sau khi đạt các tiêu chí cốt lõi	Xác định kích thước engine. Diện tích không được đánh giá vì synthesis nằm ngoài dự án.
Bảo mật / Riêng tư / An toàn	Không phải mục tiêu của dự án. Engine bỏ các command sai và ghi lại các error bit dạng sticky (err_k, err_addr, err_illegal, err_align).	Việc xử lý lỗi được đặc tả và kiểm chứng, nhưng không thực hiện phân tích bảo mật.

1.4 Câu hỏi nghiên cứu

RQ1. Một NPU RISC-V lấy cảm hứng từ CoralNPU (core RV32IM, custom instruction, MAC engine int8 tách rời, DTCM hai cổng) có thể được thiết kế để vượt qua kiểm chứng có hệ thống với 0 mismatch, 0 assertion fail và functional coverage đầy đủ hay không?

RQ2. Một môi trường UVM tự xây dựng với golden model độc lập, assertion và coverage kiểm chứng engine, core và toàn SoC hiệu quả đến mức nào, và nó có phát hiện được các lỗi được cố ý chèn vào hay không?

RQ3. NPU đã được kiểm chứng có chạy được một MLP đã lượng tử hóa (MNIST, 784-64-32-10) đầu-cuối với logit và nhãn khớp bit-exact với golden model hay không, và mỗi lần suy luận cần bao nhiêu cycle và tỉ lệ bận của engine là bao nhiêu?

Tiêu chí thành công

Tiêu chí	Thước đo	Điều kiện chấp nhận / Mục tiêu
SC1 Tính đúng chức năng	Số mismatch ở scoreboard; số assertion fail	Cả hai bằng 0 với mọi directed test và random test đã định nghĩa của engine, core và SoC
SC2 Coverage	Các bin functional coverage khai báo trong coverage plan	100 % bin đã khai báo trước các phép đo cuối (tuần 10); mọi ngoại lệ phải có lý do bằng văn bản
SC3 MNIST	Logit và nhãn của RTL so với golden model	Bit-exact trên 1,000 ảnh test đầu; lần chạy đủ 10,000 ảnh tái tạo đúng accuracy của golden
SC4 Tái lập	Câu lệnh và phiên bản được ghi lại	Một lệnh cho mỗi flow tái tạo kết quả regression, coverage và MNIST; phiên bản công cụ và seed được ghi lại

2 CÁC GIẢI PHÁP HIỆN CÓ VÀ CÔNG TRÌNH LIÊN QUAN

Xác lập hiện trạng kỹ thuật và rút ra một khoảng trống nghiên cứu có cơ sở.

2.1 Phạm vi tài liệu và chiến lược tìm kiếm

Phần tổng quan bao gồm bốn chủ đề tương ứng với các đóng góp của dự án: (i) kiến trúc bộ tăng tốc ghép một matrix engine với một host core; (ii) các NPU RISC-V mở và bộ sinh bộ tăng tốc; (iii) lượng tử hóa số nguyên và phần mềm chạy các model đã lượng tử hóa trên thiết bị nhỏ; và (iv) phương pháp kiểm chứng cần thiết để chứng minh một bộ tăng tốc là đúng. Các công trình được bình duyệt giai đoạn 2014–2021 được dùng cho bộ tăng tốc và lượng tử hóa, còn repository Coral NPU mã nguồn mở (commit 382b5c1) cùng các developer guide của nó, đọc ngày 15/10/2026, đại diện cho NPU RISC-V mở hiện nay. Các tiêu chuẩn (IEEE 1800, 1800.2 và RISC-V ISA manual) cùng hai giáo trình [13][14] xác định thuật ngữ.

2.2 Tóm tắt các công trình liên quan

Tài liệu	Vấn đề / Bối cảnh	Phương pháp / Kiến trúc	Kết quả chính	Hạn chế liên quan đến dự án này
[2] Jouppi et al. (2017)	Chi phí suy luận mạng neural ở trung tâm dữ liệu	Matrix unit systolic 256×256 gồm các MAC 8 bit với bộ nhớ on-chip do phần mềm quản lý	Báo cáo hiệu năng cao hơn khoảng 15–30× và TOPS/W cao hơn 30–80× so với CPU và GPU cùng thời trên workload sản xuất	Quy mô trung tâm dữ liệu và chức năng cố định; không có host RISC-V, không có RTL mở hay flow kiểm chứng
[3] Chen et al. (2017)	Năng lượng bị chi phối bởi di chuyển dữ liệu trong bộ tăng tốc CNN	Dataflow row-stationary trên 168 PE, công nghệ CMOS 65 nm	Hiệu quả năng lượng đo trên silicon nhờ tái sử dụng dữ liệu ở các lớp convolution của AlexNet	Chip CNN chức năng cố định; không có giao diện tập lệnh tới host core; không có RTL mở hay flow kiểm chứng
[7] Genc et al. (2021)	Đánh giá bộ tăng tốc DNN cùng với hệ thống host của chúng	Bộ sinh Chisel cho bộ tăng tốc systolic-array gắn vào core RISC-V, kèm software stack	Khảo sát không gian thiết kế toàn hệ thống giữa bộ tăng tốc và host	Hướng tới khảo sát không gian thiết kế chứ không phải một thiết kế tham chiếu nhỏ, được kiểm chứng đầy đủ; code base lớn
[6] Google, Coral NPU	NPU RISC-V mở cho edge AI	Scalar core in-order, vector unit và matrix unit outer-product; thường dùng MAC array 8×8×4; có test UVM và cocotb	Hệ thống mở hoàn chỉnh có developer guide và các workload ML mẫu	Code base Chisel lớn; matrix unit được đánh dấu thử nghiệm trong repository; khó nghiên cứu trọn vẹn đầu-cuối trong một học kỳ
[1] Jacob et al. (2018)	Chạy mạng neural chỉ bằng số học số nguyên	Weight và activation int8, tích lũy int32, requantization dấu phẩy cố định	Cải thiện đánh đổi accuracy–độ trễ cho suy luận MobileNet trên CPU di động	Là phần mềm trên CPU; việc thực thi trên phần cứng và golden model bit-exact cho phần cứng để lại cho người hiện thực
[15] David et al. (2021)	Chạy model ML trên vi điều khiển	Framework dạng interpreter với các kernel khả chuyển cho thiết bị nhúng	Framework được dùng rộng rãi cho thiết bị nhỏ	Chỉ là phần mềm; không có custom instruction hay engine chuyên dụng; flow compiler nằm ngoài dự án này

2.3 So sánh phản biện các hướng tiếp cận hiện có

Hướng tiếp cận	Ý tưởng chính	Ưu điểm	Hạn chế	Điều kiện đánh giá / Hiệu năng
Hướng A Bộ tăng tốc dataflow và systolic lớn ([2][3])	Tối đa hóa mức sử dụng và tái sử dụng dữ liệu bằng một array và dataflow chuyên dụng	Thông lượng trên mỗi watt cao khi array bận	Chức năng cố định và lớn; không có flow kiểm chứng mở; kích thước vượt quá một dự án hai người	ASIC, node và workload khác nhau; các con số không so sánh trực tiếp được với nhau hay với dự án này

Hướng tiếp cận	Ý tưởng chính	Ưu điểm	Hạn chế	Điều kiện đánh giá / Hiệu năng
Hướng B Bộ sinh bộ tăng tốc trên RISC-V ([7])	Sinh các systolic array cấu hình được, gắn vào core RISC-V	Linh hoạt, có software stack và đánh giá toàn hệ thống	Tập trung vào khảo sát không gian thiết kế; một tham chiếu nhỏ được kiểm chứng với việc chạy model bit-exact không phải mục tiêu của nó	Benchmark toàn hệ thống trên FPGA hoặc mô phỏng; không so sánh trực tiếp được với MLP MNIST
Hướng C NPU RISC-V mở ([6])	Scalar core cùng matrix engine tách rời với bộ nhớ gắn chặt	Hệ thống mở hoàn chỉnh có software stack; có cấu trúc UVM để tham khảo	Code base rất lớn; matrix unit được đánh dấu thử nghiệm; không được thu nhỏ để một nhóm nhỏ nghiên cứu đầy đủ	Developer guide và tài liệu thiết kế; không so sánh trực tiếp được với số cycle của dự án này
Hướng D Suy luận lượng tử hóa bằng phần mềm ([1][15])	Datapath int8 với accumulator int32 và rescale dấu phẩy cố định, chạy bởi một runtime phần mềm	Giữ được accuracy; chạy được trên vi điều khiển có sẵn	Không có engine chuyên dụng; việc thực thi trên phần cứng và kiểm chứng nó nằm ngoài phạm vi	MobileNet trên ImageNet chạy trên CPU di động, benchmark vi điều khiển; không so sánh trực tiếp được với MLP MNIST trên NPU riêng

2.4 Khoảng trống nghiên cứu được xác định

Các thiết kế hiện có hoặc là bộ tăng tốc chức năng cố định quy mô lớn [2][3], hoặc là bộ sinh nhằm khảo sát không gian thiết kế [7], hoặc là một NPU mở lớn mà matrix engine vẫn còn thử nghiệm [6]; còn các software stack [1][15] chạy model đã lượng tử hóa trên vi điều khiển mà không có engine chuyên dụng. Trong các nguồn đã xem xét ở đây, không có công trình nào cung cấp một NPU nhỏ, được tài liệu hóa đầy đủ, kết hợp cùng lúc ba yếu tố mà dự án này cần: (a) một giao diện custom-instruction từ một core tự xây tới một engine outer-product int8 tách rời; (b) kiểm chứng bằng một môi trường UVM với golden model độc lập, assertion và functional coverage được đóng; và (c) một lần chạy đầu-cuối của một model đã lượng tử hóa mà kết quả khớp bit-exact với golden model.

Đây là khoảng trống về các thiết kế tham chiếu được kiểm chứng và tái lập được, không phải tuyên bố về một kiến trúc mới. Kiến trúc đi theo Coral NPU [6]; đóng góp là một thiết kế mà một nhóm nhỏ có thể kiểm chứng trọn vẹn và dùng để chạy một model thật.

2.5 Tuyên bố định vị

So với các hướng tiếp cận đã xem xét, dự án này không đề xuất một bộ tăng tốc lớn hơn [2][7] hay một dataflow mới [3]. Đóng góp của nó là một nghiên cứu quy mô nhỏ nhưng hoàn chỉnh và tái lập được: một NPU được thiết kế từ tập lệnh xuống RTL theo ngữ nghĩa của Coral NPU [6] (engine outer-product N×M×K, PE dot4, hàng đợi command, bộ nhớ gắn chặt), được kiểm chứng bằng một môi trường UVM tự xây dựng, và được chứng minh chạy một MLP MNIST đã lượng tử hóa khớp bit-exact. Nó khác [6] ở chỗ dùng core 5 tầng tự xây, một tile 4×4×4 duy nhất, các golden model độc lập bằng C và Python, và bằng chứng kiểm chứng được tái tạo bằng một lệnh cho mỗi flow, để mọi quyết định thiết kế đều giải thích được.

3 Ý TƯỞNG ĐỀ XUẤT / GIẢI PHÁP ĐỀ XUẤT

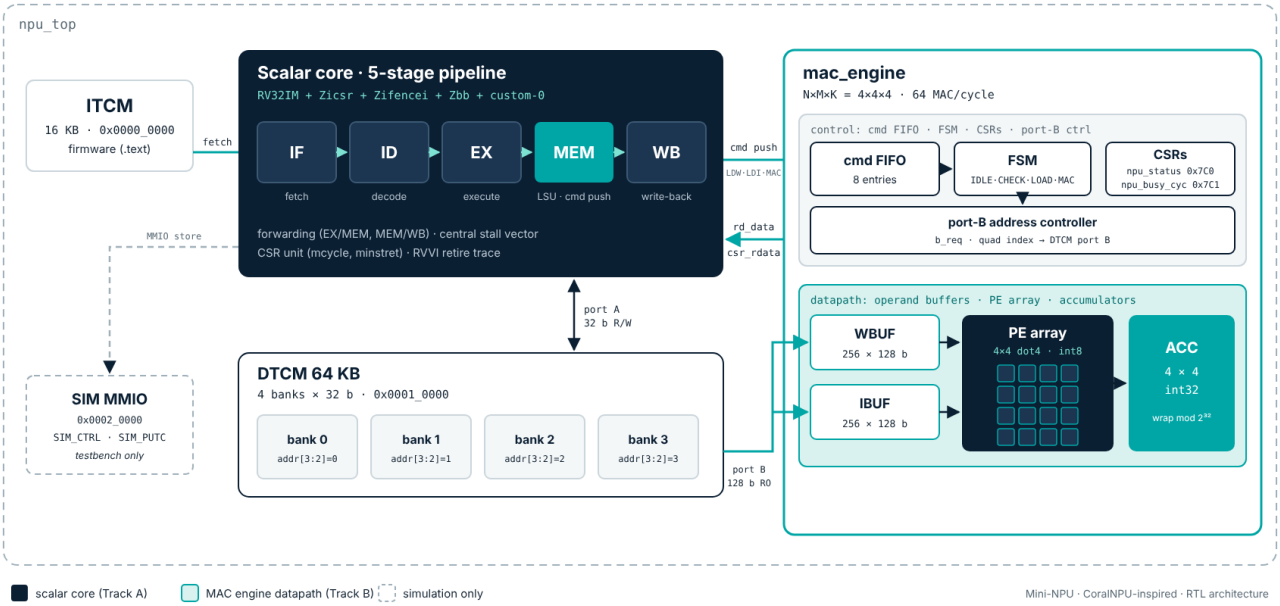
Mô tả khái niệm giải pháp, kiến trúc, tính mới và các giả thuyết kiểm chứng được.

3.1 Ý tưởng kỹ thuật cốt lõi

Dự án này giới thiệu một NPU RISC-V lấy cảm hứng từ CoralNPU, trong đó bốn custom-0 instruction đẩy command qua một FIFO 8 phần tử tới một MAC engine outer-product int8 chạy tách rời (4×4×4 = 64 MAC mỗi cycle), engine này đọc toán hạng từ một DTCM hai cổng và tích lũy bằng int32. NPU được kiểm chứng bằng ba môi trường UVM tự xây (engine, core và SoC) đối chiếu mọi kết quả với các golden model độc lập, kèm assertion và functional coverage. Khả năng của nó sau đó được chứng minh bằng cách chạy một MLP 784-64-32-10 đã lượng tử hóa trên MNIST bằng firmware bare-metal, với logit phải khớp bit-exact với golden model. Điều này giải quyết khoảng trống ở Mục 2.4: một NPU nhỏ theo kiến trúc kiểu Coral, có bằng chứng kiểm chứng có hệ thống và một lần chạy model đầu-cuối.

3.2 Kiến trúc hệ thống đề xuất

Thành phần kiến trúc	Mô tả / Giao diện / Chức năng
Đầu vào / Thu nhận dữ liệu	Firmware bare-metal (C và assembly, custom instruction qua các wrapper `.insn`) và dữ liệu MLP int8 được nạp vào ITCM và DTCM bằng backdoor `$readmemh` trước reset. Pixel MNIST p được lượng tử hóa thành (p·254 + 255) // 510 trong khoảng 0..127; model 784-64-32-10 được huấn luyện trong notebook và xuất ra file `.mem` kèm hash SHA-256.
Tiền xử lý / Lớp giao diện	Core 5 tầng RV32IM + Zicsr + Zifencei + Zbb giải mã các custom-0 instruction và đẩy các command LOAD_WEIGHT, LOAD_INPUT, COMPUTE từ tầng MEM vào command FIFO 8 phần tử; READ_RESULT stall cho đến khi engine xả hết.
Module cốt lõi đề xuất	`mac_engine`: FSM (IDLE, CHECK, LOAD, MAC); operand buffer WBUF và IBUF (256 × 128 bit); 16 PE dot4 và một tile accumulator int32 4×4 (wrap theo modulo 2^{32}); các CSR npu_status và npu_busy_cyc. Toán hạng đến từ DTCM port B (128 bit, chỉ đọc).
Tối ưu / Điều khiển / Lập lịch	Command FIFO và FSM của engine: mọi command được kiểm tra ở trạng thái CHECK (op sai, dải K và bội số của 4, căn chỉnh, dải địa chỉ); command sai bị bỏ và ghi vào các error bit dạng sticky; READ_RESULT đóng vai trò barrier chờ đến khi FIFO rỗng và engine idle. Một quy tắc phần mềm ngăn ghi đè dữ liệu DTCM mà một LOAD đang xếp hàng còn đọc.
Đầu ra / Quyết định / Thực thi	READ_RESULT trả các entry accumulator về một thanh ghi scalar; firmware cộng bias, requantize bằng clamp((acc·M + 2^(S−1)) >> S, 0, 127) và lấy argmax của 10 trong 12 logit đã đệm. Bài test kết thúc bằng một lệnh ghi vào SIM_CTRL.
Giao diện giám sát / Đánh giá	Ba môi trường UVM (mac_env, core_env, soc_env) với các golden model C gọi qua DPI-C (npu_ref và một instruction-set simulator RV32) và một model số nguyên bằng Python cho MLP, SystemVerilog assertion, functional coverage, và một agent retire-trace thụ động so sánh từng lệnh đã retire với instruction-set simulator.



Hình 1. Kiến trúc hệ thống Mini-NPU (npu_top). Scalar core (màu tối) đẩy command tới MAC engine tách rời (màu xanh ngọc), và cả hai dùng chung DTCM hai cổng 64 KB. Khối SIM MMIO chỉ tồn tại trong mô phỏng.

3.3 Tính mới và đóng góp kỹ thuật kỳ vọng

Đóng góp 1. Một NPU RISC-V quy mô nhỏ chạy được bằng RTL theo kiến trúc Coral NPU: extension custom-0 instruction, command FIFO, engine outer-product int8 tách rời và DTCM hai cổng.

Đóng góp 2. Một hạ tầng kiểm chứng UVM tự xây dựng (ba môi trường, golden model DPI-C, assertion và functional coverage) cùng bằng chứng đóng của nó, được kiểm tra độ nhạy bằng các lỗi được chèn vào.

Đóng góp 3. Một minh chứng đầu-cuối: một MLP 784-64-32-10 đã lượng tử hóa, cùng pipeline dữ liệu và firmware của nó, chạy trên NPU và khớp bit-exact với golden model trên MNIST.

Đóng góp 4. Một gói tái lập tạo lại kết quả regression, coverage và MNIST bằng một lệnh cho mỗi flow.

Tính mới nằm ở sự kết hợp và ở bằng chứng, không phải ở một kiến trúc mới (Mục 2.4).

3.4 Giả thuyết

Giả thuyết	Phát biểu kiểm chứng được	Bằng chứng cần có
H1	NPU vượt qua toàn bộ bộ kiểm chứng (engine, core và SoC) với 0 mismatch ở scoreboard, 0 assertion fail và 100 % bin coverage đã khai báo.	Log regression với số UVM_ERROR và UVM_FATAL; báo cáo coverage đã merge
H2	Các môi trường UVM có độ nhạy: mỗi lỗi được cố ý chèn vào (một self-test hoặc RTL hay testbench bị mutate) làm môi trường fail đúng theo lý do đã nêu.	Các self-test expected-fail và kết quả chèn lỗi
H3	MLP MNIST chạy trên NPU với logit và nhãn bit-exact so với golden model trên 1,000 ảnh test đầu, và lần chạy đủ tái tạo đúng accuracy của golden.	Golden dump so với LOGITS dump của RTL; accuracy trên 10,000 ảnh test

3.5 Phạm vi

TRONG PHẠM VI:

– Các extension M, Zicsr, Zifencei và Zbb của core, và bốn custom instruction cùng command FIFO của chúng.

– MAC engine (FIFO, operand buffer, PE array, accumulator, CSR), DTCM hai cổng, ITCM và simulation MMIO.
– Ba môi trường UVM, một golden model C của engine, một instruction-set simulator C và một golden model số nguyên của MLP.
– Firmware bare-metal và pipeline MNIST (huấn luyện, lượng tử hóa, đóng gói dữ liệu, firmware và phép so sánh bit-exact).

NGOÀI PHẠM VI / KHÔNG PHẢI MỤC TIÊU (để dành cho luận văn tốt nghiệp):

– Clock gating và mọi việc ước lượng hay tối ưu công suất, năng lượng; synthesis, place-and-route và bring-up trên FPGA hay board.
– RVV/Zve32x và extension F; DMA, nhiều tile accumulator và PE dạng pipeline.
– Compiler toolchain (IREE hoặc TensorFlow Lite Micro [15]) và nạp chương trình qua frontdoor (bus).
– Kiểm chứng có hỗ trợ LLM. Mảng 8×8 chỉ được xây nếu còn thời gian sau khi đạt các tiêu chí cốt lõi.

4 PHƯƠNG PHÁP LUẬN / KẾ HOẠCH HIỆN THỰC VÀ ĐÁNH GIÁ

Giải thích ý tưởng đề xuất sẽ được xây dựng, kiểm thử, so sánh và phân tích như thế nào.

4.1 Phương pháp nghiên cứu tổng thể

Giai đoạn	Hoạt động chính	Đầu ra / Bằng chứng
A. Xác định vấn đề và baseline	Chốt phạm vi; đóng băng custom ISA và interface của engine; kiểm tra công cụ (UVM, DPI-C, coverage); viết golden model của engine kèm unit test.	Spec thiết kế kèm decision log; interface đã đóng băng; kết quả kiểm tra công cụ
B. Thiết kế	Mở rộng core (M, Zicsr, Zifencei, Zbb, custom-0, retire trace); viết DTCM hai cổng, MAC engine và SoC top; viết nền firmware.	RTL; ghi chú theo khối; kết quả directed test và regression
C. Hiện thực và kiểm chứng	Xây mac_env, core_env và soc_env; viết sequence constrained-random (từ 200 seed trở lên), assertion và coverage; kiểm tra các phép check bằng cách chèn lỗi.	Log regression; báo cáo coverage đã merge (SC1, SC2); kết quả chèn lỗi
D. Minh chứng với model	Huấn luyện và lượng tử hóa MLP; đóng gói weight và input vào ảnh DTCM; viết firmware; chạy các ảnh MNIST trên NPU và so sánh với golden model.	File export kèm hash; firmware; so sánh bit-exact trên 1,000 ảnh và accuracy trên 10,000 ảnh (SC3)
E. Phân tích	Đo số cycle mỗi lần suy luận và tỉ lệ bận của engine; ghi lại hạn chế; viết báo cáo và gói tái lập.	Hình và bảng; báo cáo tiếng Anh; slide; tái tạo bằng một lệnh (SC4)

4.2 Kế hoạch hiện thực

Bước	Đầu vào	Xử lý / Hành động thiết kế	Đầu ra
1	Tài liệu CoralNPU; yêu cầu môn học	Nghiên cứu thiết kế mở; viết spec, decision log và tiêu chí thành công	Spec đã duyệt; ISA và interface đã đóng băng
2	Spec mục 6, 7, 9	Viết các golden model: npu_ref (C), một instruction-set simulator RV32 (C) và một model MLP số nguyên (Python); chặn mỗi model bằng unit test riêng	Các oracle đã qua unit test của chúng
3	Core RV32I 5 tầng có sẵn	Thêm M, Zicsr, Zifencei, Zbb, giải mã và đẩy custom-0, ITCM và DTCM, và cổng retire-trace; đối chiếu với một ISS tham chiếu bằng Python	Regression của core xanh
4	Interface engine đã đóng băng	Viết command FIFO, FSM, các trạng thái LOAD và MAC, PE array, CSR và reset; đo độ trễ theo từng command	MAC engine; directed suite xanh
5	RTL core và engine	Tích hợp chúng trong SoC top; viết test firmware	SoC smoke test
6	Kế hoạch directed và random; coverage plan	Xây các môi trường UVM; random sequence trên từ 200 seed trở lên; assertion A-MAC-1..5; đóng coverage; chèn lỗi để kiểm tra các phép check	Bằng chứng SC1 và SC2
7	MLP đã huấn luyện; NPU đã kiểm chứng	Export model; đóng gói weight và input; viết firmware MNIST; chạy 100, 1,000 và 10,000 ảnh và so sánh logit với golden model	Bằng chứng SC3; số cycle mỗi lần suy luận
8	Mọi flow	Làm cho mỗi flow chạy được bằng một lệnh; ghi lại phiên bản công cụ, seed và git hash; thử trên một bản clone sạch	Gói tái lập SC4

4.3 Nền tảng phần cứng và phần mềm

Thành phần / Công cụ	Phiên bản / Đặc tả	Mục đích trong dự án
Nền tảng xử lý	PC Windows với WSL2 (Ubuntu 24.04)	Máy chủ cho QuestaSim và cho các công cụ Linux của regression
Cảm biến / Giao diện	Không áp dụng (chỉ mô phỏng)	Không bring-up board trong dự án này

Thành phần / Công cụ	Phiên bản / Đặc tả	Mục đích trong dự án
Hệ điều hành / RTOS	Windows với WSL2 Ubuntu 24.04; CI trên Ubuntu 24.04. Không có RTOS: firmware bare-metal	Cùng một môi trường Linux ở máy cục bộ và trong continuous integration
Ngôn ngữ lập trình	SystemVerilog và Verilog-2001 (RTL, testbench); C99 (golden model, DPI-C); Python 3; assembly và C cho RISC-V (firmware, `-march=rv32im_zicsr_zifencei_zbb`)	RTL và kiểm chứng; golden model và firmware; huấn luyện và xuất dữ liệu
Framework / Thư viện	UVM 1.2 (IEEE 1800.2 [10]); numpy; PyTorch (chỉ để huấn luyện); pytest	Phương pháp kiểm chứng; tham chiếu số nguyên và unit test của nó; huấn luyện MLP
Công cụ phát triển / Mô phỏng	QuestaSim 10.7c; Verilator 5.020 (lint); Icarus Verilog 12 (regression của core); RISC-V GNU toolchain; GitHub Actions	Mô phỏng và coverage; lint; regression của core; biên dịch firmware; regression tự động ở mỗi lần push

4.4 Dữ liệu, workload, benchmark hoặc kịch bản kiểm thử

Mục	Định nghĩa
Dataset / benchmark / workload	(1) MNIST [12] với MLP 784-64-32-10 được lượng tử hóa int8, chạy bởi firmware bare-metal. (2) Các test directed và constrained-random cho engine, core và SoC.
Kích thước / số mẫu / thời lượng	MNIST: 1,000 ảnh test đầu cho phép so sánh bit-exact và toàn bộ tập test 10,000 ảnh cho accuracy; một lần chạy 100 ảnh cho phép đo cycle. Test random: từ 200 seed trở lên cho mỗi môi trường.
Tiền xử lý / chuẩn hóa / biên dịch	Pixel sang int8 bằng (p·254 + 255) // 510. Weight int8 đối xứng trong ±127, bias int32. Requantization theo từng lớp với hệ số nhân nguyên M và độ dịch S chọn là S lớn nhất thỏa bound·M + 2^(S−1) ≤ 2³¹−1. Lớp cuối được đệm từ 10 lên 12 output; argmax bỏ qua phần đệm.
Chia train / validation / test	Huấn luyện trên tập train 60,000 ảnh (seed 0); 10,000 ảnh train dùng để hiệu chỉnh scale của activation (phân vị 99.99); đánh giá trên tập test 10,000 ảnh. Các phép so sánh phần cứng chỉ dùng ảnh test.
Điều kiện vận hành / dải đầu vào	Mô phỏng RTL một miền clock, không back-annotate timing; cùng binary và phiên bản simulator cho mọi phép so sánh; seed random được đặt tường minh.
Ground truth / đầu ra tham chiếu	Các golden model độc lập (engine, instruction-set simulator, MLP số nguyên) và retire trace của core; các model chỉ được dùng sau khi qua unit test của chính chúng.

4.5 Baseline và phương pháp so sánh

Baseline	Lý do chọn	Cấu hình / Kiểm soát công bằng
Baseline 1: các golden model độc lập	Oracle cho tính đúng chức năng; so sánh bit-exact loại bỏ tranh luận về dung sai	Chỉ dùng sau khi qua unit test của chúng; đối chiếu chéo với nhau (model MLP số nguyên với model engine, instruction-set simulator với một ISS tham chiếu Python)
Baseline 2: MLP float	Tham chiếu accuracy cho model int8 [1]	Cùng weight đã huấn luyện, cùng 10,000 ảnh test

4.6 Thước đo đánh giá

Khía cạnh	Thước đo ứng viên	Thước đo được chọn và cách đo
Chất lượng / ML	Accuracy, mức khớp nhãn	Accuracy của int8 và float trên tập test; logit và nhãn khớp bit-exact với golden model
Hiệu năng	Cycle, độ trễ, thông lượng	mcycle và minstret mỗi lần suy luận; số cycle bận cho mỗi command của engine (chặn trên K/4 + 3); tỉ lệ bận của engine = npu_busy_cyc / mcycle. Được báo cáo, không đặt trước mục tiêu.

Khía cạnh	Thước đo ứng viên	Thước đo được chọn và cách đo
Tính toán / Bộ nhớ	MAC mỗi cycle, dung lượng bộ nhớ	Đỉnh 64 MAC mỗi cycle; vùng dữ liệu DTCM 56,816 B trên 65,536 B
Phần cứng	LUT, FF, diện tích, tần số	Không đánh giá (không có synthesis)
Năng lượng	Công suất, năng lượng mỗi lần suy luận	Không đánh giá (để dành cho luận văn)
Độ tin cậy	Mismatch, assertion fail, coverage	Số mismatch ở scoreboard và assertion fail (mục tiêu 0); functional coverage đã merge (mục tiêu 100 % bin đã khai báo); các lỗi chèn vào đều bị phát hiện

4.7 Thiết kế thí nghiệm

Thí nghiệm	Mục tiêu	Biến độc lập	Đại lượng đo	Quyết định / So sánh
E1: Kiểm chứng chức năng	Chứng minh engine, core và SoC đúng so với golden model	Môi trường (mac_env, core_env, soc_env); test (directed, random seed)	Số mismatch ở scoreboard; số assertion fail; coverage %	Chỉ chấp nhận khi 0 mismatch, 0 failure và 100 % bin đã khai báo (SC1, SC2)
E2: Độ nhạy của các phép check	Chứng minh các phép check phát hiện được lỗi thật	Lỗi được chèn (self-test, RTL bị mutate hoặc testbench bị mutate)	Run fail hay pass; ID lỗi được báo	Mỗi lỗi phải bị bắt đúng theo lý do đã nêu (H2)
E3: MNIST đầu-cuối	Chứng minh NPU chạy model đã lượng tử hóa bit-exact	Số ảnh (100, 1,000, 10,000)	Logit và nhãn so với golden; accuracy; mcycle và npu_busy_cyc mỗi lần suy luận	Bit-exact (SC3); accuracy bằng accuracy của golden; số cycle được báo cáo

4.8 Kế hoạch phân tích và tái lập

- Mọi lần mô phỏng chạy qua một Python runner duy nhất (có tùy chọn cho simulator, test target, seed và coverage); log được giữ theo từng lần chạy trong một thư mục theo ngày kèm `manifest.yaml` ghi lại phiên bản QuestaSim và loại licence, Verilator, GCC và Python, git hash, seed và SHA-256 của từng log.
- Seed random được đặt tường minh (ví dụ 1..200). Các golden model được chặn cổng: mỗi model phải qua unit test của chính nó trước khi dùng làm oracle, và các testbench được kiểm tra bằng cách chèn lỗi và xác nhận rằng các phép kiểm tra fail.
- Dữ liệu model được export kèm SHA-256 của từng file; golden model số nguyên, model C của engine và RTL được so sánh trên cùng các file đã export.
- Một mismatch được khoanh vùng theo từng lớp (logit khác đầu tiên, rồi activation của lớp trước) bằng các golden model, để một lần chạy fail báo được chỗ khác biệt đầu tiên xảy ra.
- Một lệnh cho mỗi flow tái tạo kết quả regression, coverage và MNIST; điều này được kiểm trên một bản clone sạch ở các tuần cuối (SC4).

4.9 Rủi ro kỹ thuật và giảm thiểu

Rủi ro	Tác động	Giảm thiểu / Phương án dự phòng
Licence của simulator thiếu một tính năng (UVM, randomize, covergroup, DPI-C)	Flow UVM bị chặn	Đã kiểm ở tuần 1: QuestaSim 10.7c chạy được cả bốn. Dự phòng: Vivado xsim (khi đó đã xác minh), Verilator hoặc pyuvm.
Mô phỏng quá chậm cho các lần chạy MNIST	SC3 không chạy xong	Đo sớm. Thứ tự cắt giảm: lần chạy 10,000 ảnh trước, sau đó giảm số ảnh cho phép đo cycle; lần chạy bit-exact trên 1,000 ảnh và báo cáo không bao giờ bị cắt.
Interface giữa hai track thay đổi	Làm lại ở cả hai phía	Interface đóng băng ở tuần 2 (hai sinh viên đã ký ngày 29/10/2026); thay đổi sau đó cần cả hai chữ ký và cập nhật spec.

Rủi ro	Tác động	Giảm thiểu / Phương án dự phòng
Golden model và RTL cùng hiểu sai spec	Kết quả sai bị chấp nhận	Các model độc lập bằng C và Python đối chiếu chéo với nhau; chèn lỗi vào testbench; các case unit tính tay.
Firmware ghi đè dữ liệu DTCM mà một LOAD đang xếp hàng còn đọc	Dữ liệu đầu vào hỏng	Quy tắc phần mềm (xả bằng READ_RESULT trước khi ghi đè) cộng một assertion và một negative test chứng minh assertion hoạt động.
Model đã lượng tử hóa khác với requantization trong firmware	MNIST không bit-exact	Công thức requantization được cố định trong spec; routine của firmware được unit test trên host so với golden model trước khi chạy trên NPU.
Công sức kiểm chứng vượt quá tiến độ	Coverage không đóng kịp	Directed test trước, random và coverage sau; ngoại lệ coverage cần lý do bằng văn bản; tuần 13–14 là thời gian đệm; mảng 8×8 bị bỏ trước.

5 KẾT QUẢ KỲ VỌNG

Xác định các kết quả đo được, đóng góp kỳ vọng và sản phẩm bàn giao cụ thể của dự án.

5.1 Kết quả kỹ thuật kỳ vọng

NPU được kỳ vọng vượt qua toàn bộ bộ kiểm chứng với 0 mismatch ở scoreboard và 0 assertion fail, và đóng 100 % bin functional coverage đã khai báo cho engine, core và SoC. Các phép check kiểm chứng được kỳ vọng phát hiện mọi lỗi được cố ý chèn vào. MLP int8 784-64-32-10 được kỳ vọng cho logit và nhãn khớp bit-exact với golden model trên 1,000 ảnh test đầu, và tái tạo đúng accuracy của golden (97.43 % với model hiện tại) trên lần chạy đủ 10,000 ảnh. Số cycle mỗi lần suy luận và tỉ lệ bận của engine được đo và báo cáo, không đặt trước mục tiêu; chúng cho thấy thời gian chạy chia như thế nào giữa engine và firmware scalar.

5.2 Bảng hiệu năng mục tiêu

Thước đo	Baseline / Giá trị hiện tại	Mục tiêu kỳ vọng	Điều kiện đo
Thước đo chính: tính đúng chức năng	mac_env: 224/224 lần chạy đúng kỳ vọng, 81,332 phép so sánh ở scoreboard, 0 mismatch (log dự án, 10/11/2026)	0 mismatch và 0 assertion fail cho engine, core và SoC	QuestaSim 10.7c, UVM 1.2, runner; E1
Thước đo phụ: functional coverage	mac_env: 100 % của 7 covergroup đã merge (log dự án, 10/11/2026); core_env và soc_env chưa đo	100 % bin đã khai báo cho cả ba môi trường	Báo cáo coverage đã merge; E1
Thước đo tài nguyên: accuracy MNIST	Accuracy int8 trên tập test 97.43 % (float 97.48 %) trên 10,000 ảnh	Logit và nhãn của RTL khớp bit-exact trên 1,000 ảnh đầu; lần chạy đủ tái tạo accuracy của golden	Golden dump so với LOGITS dump của RTL; E3
Thước đo độ vững: độ nhạy của các phép check	Đã có self-test cho phép check memory-mirror và write-after-read trong mac_env	Mọi lỗi chèn vào đều bị bắt đúng theo lý do đã nêu	Các lần chạy chèn lỗi; E2
Thước đo tái lập	Có manifest cho từng lần chạy; một lệnh cho regression của engine	Một lệnh cho mỗi flow, được kiểm trên một bản clone sạch	Các tuần cuối; SC4

5.3 Đóng góp khoa học / kỹ thuật kỳ vọng

- Một NPU RISC-V nhỏ, được giải thích đầy đủ, theo kiến trúc Coral NPU (extension custom instruction, engine int8 tách rời, DTCM hai cổng), với các quyết định thiết kế được ghi lại.
- Một môi trường kiểm chứng UVM tự xây dựng với golden model độc lập, assertion, coverage đã đóng, và bằng chứng rằng các phép check của nó phát hiện được lỗi.
- Một minh chứng đầu-cuối rằng một MLP đã lượng tử hóa chạy trên NPU bit-exact, gồm cả pipeline dữ liệu và firmware.
- Một gói tái lập và thiết kế được tài liệu hóa, có thể làm nền cho các công việc sau như luận văn tốt nghiệp.

5.4 Sản phẩm bàn giao kỳ vọng

Sản phẩm	Nội dung tối thiểu / Bằng chứng chấp nhận
Đề cương nghiên cứu	Vấn đề, khoảng trống, ý tưởng, phương pháp luận và kết quả kỳ vọng đã được duyệt
Tổng quan tài liệu	Tổng hợp có phản biện các công trình liên quan (Phần 2) và phần công trình liên quan của báo cáo cuối
Sản phẩm thiết kế	Spec thiết kế kèm decision log, ghi chú theo khối cho core và MAC engine, hình kiến trúc và các định nghĩa interface
Prototype	RTL của core, MAC engine, DTCM hai cổng và SoC top; firmware; golden model

Sản phẩm	Nội dung tối thiểu / Bằng chứng chấp nhận
Gói thí nghiệm	Các môi trường UVM, danh sách test, runner, log kèm manifest, báo cáo coverage và các file so sánh MNIST
Báo cáo kỹ thuật	Báo cáo tiếng Anh với các chương kiểm chứng và MNIST, thảo luận hạn chế và kết luận
Thuyết trình cuối và demo	Regression trực tiếp, suy luận MNIST trên NPU kèm so sánh bit-exact, và kết quả coverage
Repository	Repository GitHub chứa RTL, testbench, model, firmware, script, tài liệu và gói tái lập

5.5 Diễn giải các kết quả âm tính hoặc hỗn hợp

- Nếu một số bin coverage không thể đạt được, mỗi bin được phân tích và hoặc được đóng bằng một test mới, hoặc được loại trừ kèm lý do bằng văn bản; phân tích đó cho thấy hành vi nào mà đặc tả khiến không thể đạt được và tự nó là một kết quả kiểm chứng.
- Nếu lần chạy MNIST lúc đầu chưa bit-exact, phép so sánh theo từng lớp khoanh vùng khác biệt đầu tiên vào engine, requantization hay đóng gói dữ liệu; một nguyên nhân gốc được ghi lại vẫn là kết quả hợp lệ và một regression test được thêm cho nó.
- Nếu số cycle mỗi lần suy luận cao, sự phân chia giữa cycle bận của engine và cycle firmware scalar cho thấy thời gian nằm ở đâu, đây là bằng chứng hữu ích cho công việc sau trên NPU (ví dụ chuyển requantization xuống phần cứng).
- Mọi kết quả đều đến từ mô phỏng RTL của một thiết kế và một simulator. Chúng là bằng chứng về thiết kế này, không phải tuyên bố về silicon hay về các NPU nói chung.

Tiến độ dự án và các mốc

Ngày tháng theo lịch của dự án: tuần 1 bắt đầu thứ Hai 12/10/2026 và tuần 16 kết thúc Chủ nhật 31/01/2027. Trạng thái tính đến 11/11/2026 (tuần 5).

Tuần	Hoạt động	Mốc / Sản phẩm bàn giao (trạng thái 11/11/2026)
1 12–18/10	Tool gate: QuestaSim UVM, DPI-C, coverage, toolchain; xác định phạm vi vấn đề	Kết quả tool gate. *Đã xong 18/10/2026.*
2 19–25/10	Tài liệu ISA và CSR; interface của engine; golden model của engine kèm unit test	ISA và interface đã đóng băng. *Golden model xong 17/10/2026; interface được hai bên ký 29/10/2026.*
3–4 26/10–8/11	M, Zicsr, Zifencei, Zbb, giải mã và đẩy custom-0; instruction-set simulator; MAC engine; DTCM hai cổng	Kết thúc Phase 1. *Regression core 1,291/1,291; directed suite của engine xanh trên QuestaSim (5/11/2026); còn một quyết định chính sách của core (W2-1) chưa chốt.*
5–6 9–22/11	Tích hợp SoC và nền firmware; directed test cho mac_env và core_env	Soc test đầu tiên. *Tích hợp phía engine với 8 chương trình assembly đã pass (10/11/2026); mac_env directed và random xong (7/11/2026).*
7–9 23/11–13/12	Test random và coverage; test soc; kiểm tra bằng chèn lỗi; đóng coverage	SC1 và SC2; bản nháp chương kiểm chứng. *mac_env: 100 % của 7 covergroup (10/11/2026); core_env và soc_env còn chờ.*
10 14–20/12	Pipeline MNIST: đóng gói weight và input, golden file, firmware	Ảnh DTCM đã đóng gói; những ảnh MNIST đầu tiên chạy trên NPU
11 21–27/12	MNIST trên 1,000 ảnh test đầu; debug theo từng lớp	SC3 (bit-exact trên 1,000 ảnh)
12 28/12–3/1	Chạy đủ 10,000 ảnh; số cycle mỗi lần suy luận và tỉ lệ bận của engine	Tái tạo accuracy của golden; bảng hiệu năng

Tuần	Hoạt động	Mốc / Sản phẩm bàn giao (trạng thái 11/11/2026)
13–14 4–17/1	Thời gian đệm cho sửa coverage hoặc MNIST; chương báo cáo kiểm chứng và MNIST; tái tạo trên bản clone sạch	SC4; bản nháp các chương
15 18–24/1	Báo cáo và slide; regression cuối	Bản nháp báo cáo và slide
16 25–31/1	Kiểm tra cuối, demo và nộp bài	Báo cáo cuối, demo và repository

Tài liệu tham khảo

[1] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704–2713.

[2] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a Tensor Processing Unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017, pp. 1–12.

[3] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[5] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, 2014, pp. 10–14.

[6] Google, "Coral NPU," GitHub repository *google-coral/coralnpu*, commit 382b5c1, and Coral NPU developer guides (developers.google.com/coral/guides), read Oct. 15, 2026.

[7] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2021, pp. 769–774.

[8] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, Dec. 2019.

[9] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Commun. ACM*, vol. 62, no. 2, pp. 48–60, Feb. 2019.

[10] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2017, 2017.

[11] *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[13] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2021.

[14] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. Pearson, 2019.

[15] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 3, 2021, pp. 800–811.